

CCNA Quiz Bot Documentation

A Discord Bot for BTS Cloud Computing Students

Aristide KATAGARUKA

Lycée Guillaume Kroll

PYTWO - Cloud Computing

May 3, 2025

Contents

1	Introduction	2
1.1	Problem Statement	2
1.2	Objective	2
2	Features	2
3	Implementation	2
3.1	Technology Stack	2
3.2	Hosting Details	3
3.3	Core Algorithm	3
3.4	Project Structure	4
4	Setup Instructions	4
4.1	Prerequisites	4
4.2	Installation	4
5	Testing	5
5.1	Testing Process	5
5.2	User Feedback	5
6	Challenges	6
7	Conclusion	6

1 Introduction

1.1 Problem Statement

BTS Cloud Computing students struggle to achieve scores above 80% on CCNA 1, 2, and 3 theory tests on netcad.com. Key challenges include:

- Lack of structured revision.
- Difficulty understanding networking concepts.
- Limited practice with multiple-choice questions (MCQs).
- Insufficient personalized feedback on weak areas.

1.2 Objective

The CCNA Quiz Bot is a Python-based Discord bot designed to enhance CCNA test preparation through structured quizzes, automated study tools, gamification, and educational resources. It aims to improve student performance and engagement.

2 Features

The bot implements the following features:

- **Quiz System:**
 - `$ccna <level>`: Single MCQ in a channel (levels 1, 2, 3).
 - `$quiz dm <level>`: 5-question private quiz via DMs.
 - `$test <level>`: 5-question simulated CCNA test.
- **Leaderboard:** `$leaderboard` displays top scorers via a Django API.
- **Study Resources:** `$resources` provides links to Cisco materials and YouTube tutorials.
- **Networking Explanations:** `$help <topic>` explains concepts (e.g., subnetting, OSI, DNS).
- **Study Sessions:** Flashcards with Pomodoro technique, posted every 4 hours.
- **Daily Quizzes:** Random CCNA question posted every 24 hours.
- **Motivational Messages:** Sent every 6 hours to encourage studying.
- **Study Groups:** `$assign_groups` assigns students based on weak topics.

3 Implementation

3.1 Technology Stack

- **Programming Language:** Python 3.8+

- **Libraries:**
 - `discord.py`: Discord API interactions.
 - `requests`: API calls for questions.
 - `django`: Score management.
 - `PIL`: Flashcard image generation.
 - Others: `asyncio`, `python-dotenv`, `asgiref`, `os`, `io`, `random`.
- **Database:** Django with Score model (`discord_id`, `name`, `point`).
- **API:** Questions from <https://polar-forest-95759-e6c7774f6065.herokuapp.com/api/random/>.
- **Hosting:** Heroku (app: `polar-forest-95759`, US region, heroku-24 stack, 1 web dyno with Gunicorn, \$7/month).

3.2 Hosting Details

The bot is hosted on Heroku under the app `polar-forest-95759`:

- **Region:** United States.
- **Stack:** heroku-24.
- **Dyno:** 1 web dyno running `gunicorn ccnaquizbot.wsgi`.
- **Memory Usage:** Average 18.3% (93.6 MB), peak 15.4% (78.7 MB).
- **Response Time:** Median 5ms, 95th percentile 247ms.
- **Deployment:** Heroku Git, latest build on May 3, 2025, by `katar711@school.lu`.

3.3 Core Algorithm

The `ccna_quiz` command drives the quiz system:

- Fetches a question via API for the specified level.
- Creates a Discord embed with question, options, and optional image.
- Awaits a numeric response within 60 seconds.
- Updates scores asynchronously using Django and provides feedback.

Listing 1: Core Quiz Logic (`ccna_quiz`)

```

1 @bot.command(name='ccna')
2 async def ccna_quiz(ctx, level: int = None):
3     if level not in [1, 2, 3]:
4         await ctx.send("Invalid level. Please specify level 1, 2, or 3.")
5         return
6     qs, answer, explanation, image_url = get_question(level)
7     embed = discord.Embed(title="Network Question", description=
        qs, color=discord.Color.blue())

```

```

8     if image_url and image_url.lower().startswith(('http://', '
        https://')):
9         embed.set_image(url=image_url)
10        question_message = await ctx.send(embed=embed)
11    def check(m):
12        return m.author == ctx.author and m.content.isdigit() and
            m.channel == ctx.channel
13    try:
14        guess = await bot.wait_for('message', check=check,
            timeout=60.0)
15        if int(guess.content) == answer:
16            points = 10
17            await update_user_score(ctx.author.id, ctx.author.
                name, points)
18            await ctx.send(f'👉Correct!👉You👉earned👉{points}👉points
                .\n👉**Explanation:**👉{explanation}')
19        else:
20            await ctx.send(f'👉Incorrect.👉The👉right👉answer👉was👉{
                answer}.\n👉**Explanation:**👉{explanation}')
21    except asyncio.TimeoutError:
22        await ctx.send('👉Timeout!')

```

3.4 Project Structure

- bot.py: Main bot logic and commands.
- ccnaquizbot/: Django project for score management.
- .env: Stores DISCORD_TOKEN, channel IDs.
- requirements.txt: Lists dependencies.

4 Setup Instructions

4.1 Prerequisites

- Python 3.8+.
- Discord API token (<https://discord.com/developers/applications>).
- Django project with SQLite database.

4.2 Installation

1. Clone the repository:

```

git clone https://github.com/Aris-Kata26/CCNA_Quiz_Bot.git
cd CCNA_Quiz_Bot

```

2. Install dependencies:

```
pip install -r requirements.txt
```

3. Configure `.env`:

```
DISCORD_TOKEN=your_discord_token  
DAILY_QUIZ_CHANNEL_ID=your_channel_id  
STUDY_SESSION_CHANNEL_ID=your_channel_id  
REMINDERS_STUDY_GROUP_CHANNEL_ID=your_channel_id  
RESOURCES_CHANNEL_ID=your_channel_id
```

4. Run Django migrations:

```
python manage.py migrate
```

5. Deploy to Heroku or run locally:

```
python bot.py
```

5 Testing

5.1 Testing Process

The bot was rigorously tested with at least two BTS students in a Discord server:

- **Unit Testing:**
 - Commands: Verified functionality of `$resources`, `$help`, `$quiz`, etc., with edge cases (e.g., invalid levels).
 - API: Tested `get_question` for valid responses and error handling.
 - Database: Ensured `update_user_score` correctly updates scores.
- **Functional Testing:**
 - Confirmed motivational messages, daily quizzes, and study sessions operate as scheduled.
 - Verified private quizzes and study group assignments.
- **Integration Testing:** Ensured seamless interaction between Discord, external API, and Django backend.
- **Permission Testing:** Validated bot permissions for sending messages and handling DMs.
- **Stress Testing:** Simulated multiple concurrent users to confirm stability.

5.2 User Feedback

- **Positive:**

- Intuitive commands and clear `$help` instructions.
- High engagement with motivational messages and leaderboards.
- Comprehensive learning via quizzes and explanations.
- Private quizzes appreciated for focused practice.
- **Constructive:**
 - Request for customizable message frequency.
 - Desire for custom quizzes and gamification (e.g., badges).
 - Occasional API timeout issues noted.
 - Suggestion for multi-language support.

6 Challenges

Development faced the following challenges:

- **API Integration:** Handled unreliable API responses with try-except blocks and debug logs.
- **Concurrent Interactions:** Used `asyncio` and user-specific checks for private quizzes.
- **Permissions:** Removed channel restrictions and ensured proper bot permissions.
- **Error Handling:** Improved user-facing error messages for clarity.
- **Time Management:** Prioritized core features within the timeline.
- **User Engagement:** Added motivational messages and leaderboards.
- **Scalability:** Optimized Django ORM and async tasks for growth.

7 Conclusion

The CCNA Quiz Bot successfully enhances CCNA preparation for BTS students through interactive quizzes, automated tools, and gamification. User feedback confirms its effectiveness, with future potential for customization, gamification, and multi-language support.